

«C# разработчик. Экспертный уровень»

Организация памяти в .NET: Stack, Heap, LOH и GC





Напишите «+» в чат, если меня слышно и видно





Нилов Павел

Fullstack-разработчик

Преподаватель курсов “C# Developer Basic”, “C# Developer Professional”, “ASP.NET” в ОТУС

 t.me/NilovPavel

 nilowpavel@gmail.com

Правила вебинара



Активно участвуем. Можно врываться голосом



Задаем вопросы в чат



Вопросы вижу в чате,
могу ответить не сразу

Давайте познакомимся



Какая у вас актуальная
должность?

С какой основной целью пришли на курс?

Маршрут вебинара

1. Память процесса

2. Ссылочные и значимые типы

3. Stack

4. Managed Heap

5. Упаковка и распаковка

6. Сборщик мусора, алгоритм работы

7. LOH и его особенности

8. POH



Цели вебинара

К концу занятия вы сможете

Узнать из каких сегментов памяти
состоит приложение

Узнать, как размещаются переменные в
приложении .NET

Узнать на какие части разделена куча
в .NET и их особенности

Смысл

Для чего вам это
уметь

Полученные знания позволят более
глубоко понимать внутреннее
устройство .NET-приложений и
некоторые нюансы разработки.

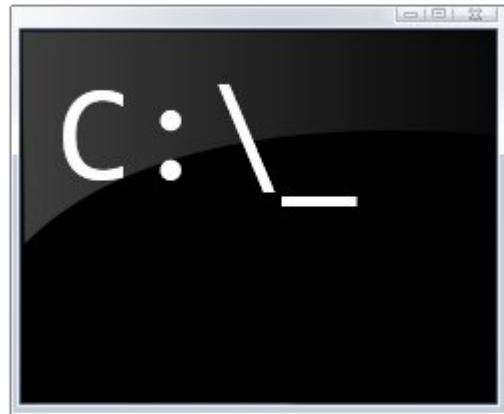
Память процесса



Что такое процесс?

Процесс в операционной системе – это работающая программа, включающая в себя инструкции кода, данные и системные ресурсы, необходимые для её выполнения (например, память и процессорное время).

Также процесс можно охарактеризовать как исполняемый экземпляр пользовательской программы.

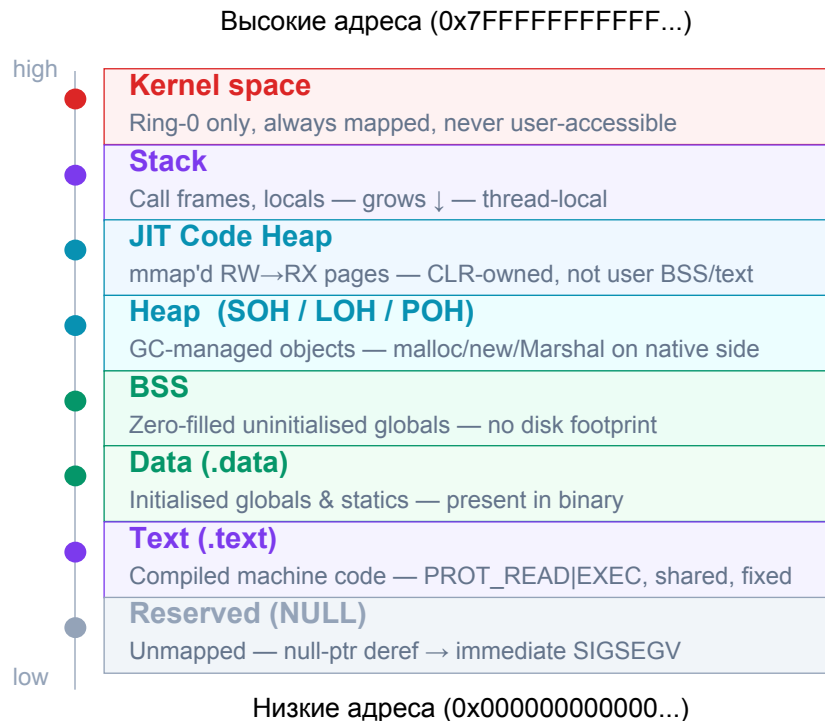


Сегменты памяти процесса

Память процесса условно делится на сегменты

1. Код программы (неуправляемая память).
2. Статическая память.
3. Управляемая куча.
4. *Стек.

**стек создается для каждого потока - свой.*

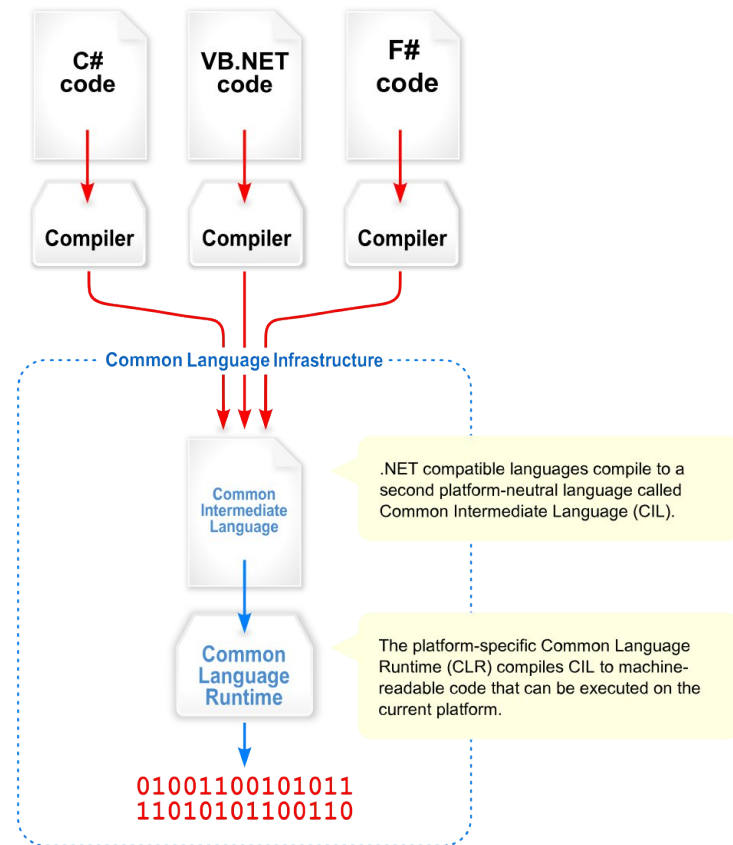


Неуправляемая память

Неуправляемая память (native memory) — это память, которую процесс CLR выделяет напрямую у операционной системы, но которая находится за пределами контроля сборщика мусора.

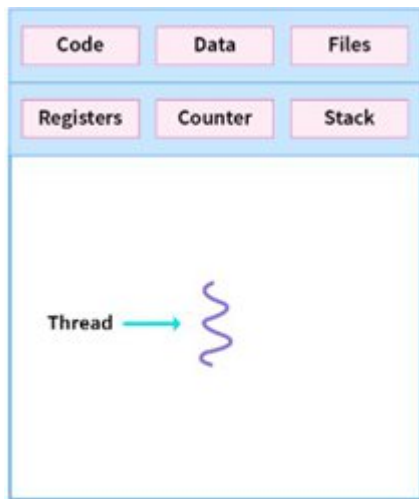
Она используется для хранения:

1. Внутренних структур данных самой CLR (метаданные классов).
2. Ресурсов операционной системы, таких как файловые дескрипторы, сетевые сокеты и подключения к базам данных.
3. Данных, выделенных вручную при использовании взаимодействия небезопасным кодом (**unsafe** code).

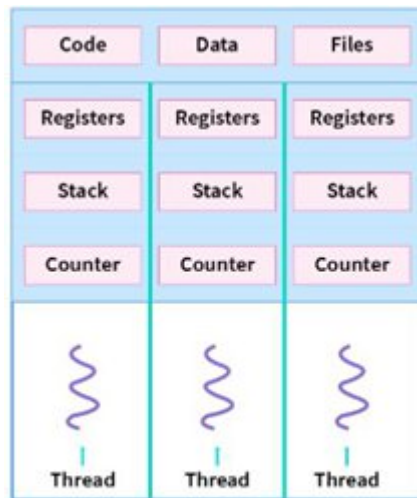


Многопоточные процессы

В большинстве случаев процессы являются однопоточными. В отдельных случаях программист может создать и управлять несколькими потоками в рамках одного процесса.



Single-threaded process



Multithreaded process

Итого

Выделение памяти под процесс:

1. Неуправляемая память
2. Данные процесса
3. Стек приложения
4. Куча приложения
5. При создании потока в приложении создается отдельный стек.



Ссылочные и значимые типы



Значимые типы

Значимые типы – это типы, которые хранят непосредственно значения данных.

Значимые типы помечаются следующими ключевыми словами:

1. Все примитивные типы: `bool`, `byte`, `sbyte`, `short`, `ushort`, `int`, `uint`, `long`, `ulong`, `float`, `double`, `decimal`, `char`
2. `record` `struct`
3. `struct`
4. `enum`

```
int value = 1000;
```

int

value (1000)

Особенности значимых типов

1. Потомки `System.ValueType`;
2. Значимые типы не наследуются; Они не могут иметь виртуальные и абстрактные методы. Методы являются неявно запечатанными - их нельзя переопределить;
3. Переменные значимых типов находятся в стеке;
4. При «копировании» объекта копируется значение полей объекта;
5. Неупакованные значимые типы удаляются из стека сразу, когда будет возвращено управление методу, их вызывающему.

Есть особенности значимых типов, которые отражаются на работе компилятора:

1. `System.ValueType` - наследник `System.Object`. Однако при этом не является ссылочным, так как компилятор "знает", что тип значимый и размещает объект в стеке.
2. Поскольку значимые типы также могут иметь методы, но ссылки на **объект-тип** у них нет, как у ссылочных, а следовательно и на таблицу методов нет, поэтому компилятор ищет объект тип в неуправляемой памяти CLR напрямую.
3. Методы `Equals` и `GetHashCode` переопределены для значимых типов и завязаны на значения полей. `Equals` сравнивает значения полей, а `GetHashCode` считается по алгоритму, учитывающему значения полей.

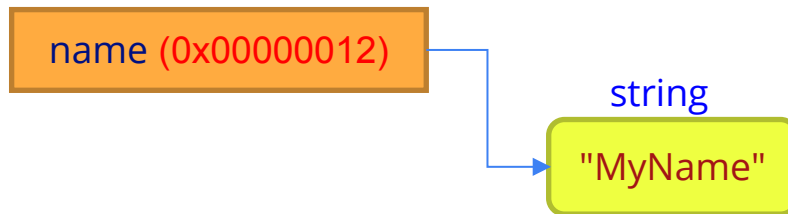
Ссылочные типы

Ссылочные типы – это типы, которые хранят ссылку на область памяти, где находится значение.

Ссылочные типы помечаются следующими ключевыми словами:

1. `class`
2. `interface`
3. `delegate`
4. `object`
5. `record`

```
string name = "MyName";
```



Особенности ссылочных типов

1. Память для ссылочных типов всегда выделяется из управляемой кучи;
2. Каждый объект, размещаемый в куче, содержит дополнительные члены, подлежащие инициализации;
3. Незанятые полезной информацией байты объекта обнуляются (это касается полей);
4. Размещение объекта в управляемой куче со временем инициирует сборку мусора;
5. Неинициализированные переменные ссылочного типа могут быть **null**, то есть ни на что не ссылаться;
6. При «копировании» объекта копируется только адрес объекта;
7. Ввиду предыдущего утверждения, несколько участков кода могут работать с одним объектом ссылочного типа и могут обновить его содержимое.

Отличия ссылочных и значимых типов

Признак	Ссылочный тип (class)	Значимый тип (struct)
Наследуется от	<code>System.Object</code>	<code>System.ValueType</code> → <code>System.Object</code>
Где хранится	В куче (heap)	В стеке
Поведение при присваивании	По ссылке	Копируется
Может быть null	Да	Нет
Поддерживает наследование	Да	Нет
Boxing/unboxing нужен	Нет, ссылочные типы всегда упакованы	Да

Stack: модели хранения и жизненный цикл объектов

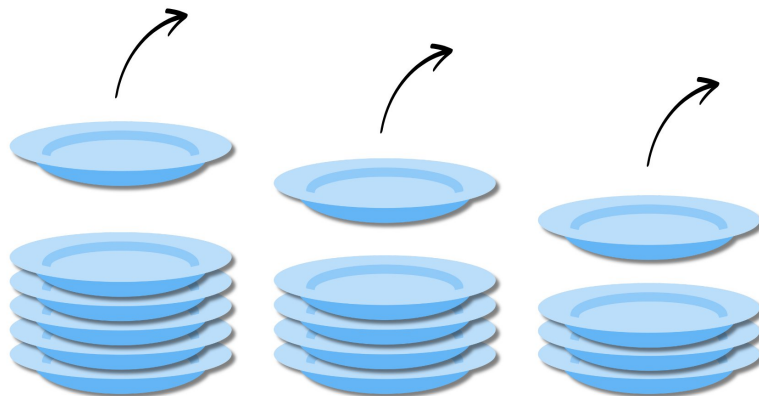


Что такое стек?

В контексте программирования, **стек (stack)** - это структура данных, которая работает по принципу "последним пришел, первым вышел" (LIFO - Last In, First Out).

Кадр стека — это область памяти в стеке потока, которая выделяется при вызове функции (метода) и хранит: параметры функций, локальные переменные, адрес возврата в функцию

Переполнение стека побуждает выброс исключения [StackOverflowException](#).



Как работает Stack?

LIFO:

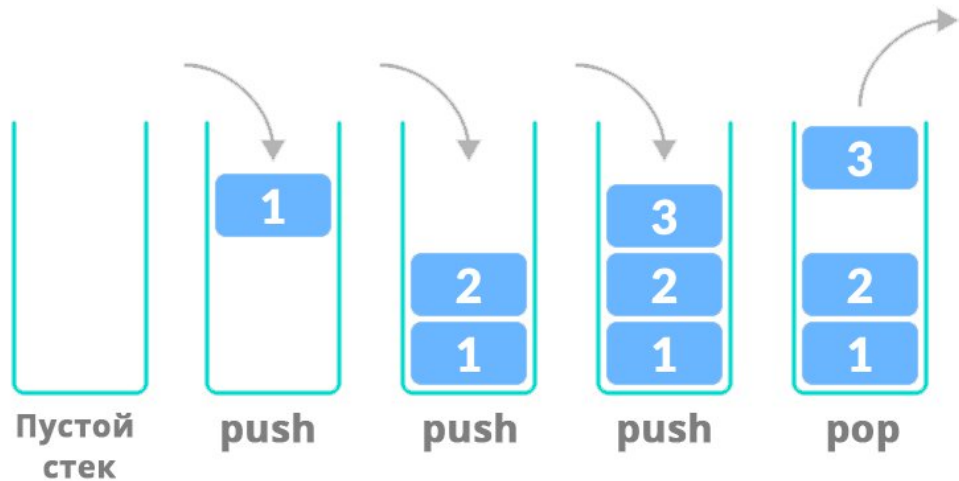
Стек гарантирует, что элемент, добавленный в него последним, будет первым извлечен. Это основной принцип работы стека.

Операции:

Две основные операции со стеком:

push: Добавление элемента в вершину стека.

pop: Извлечение элемента с вершины стека.



Стек растет в сторону уменьшения адресов.

Как работает Stack?

```
string name = "Joe";
```

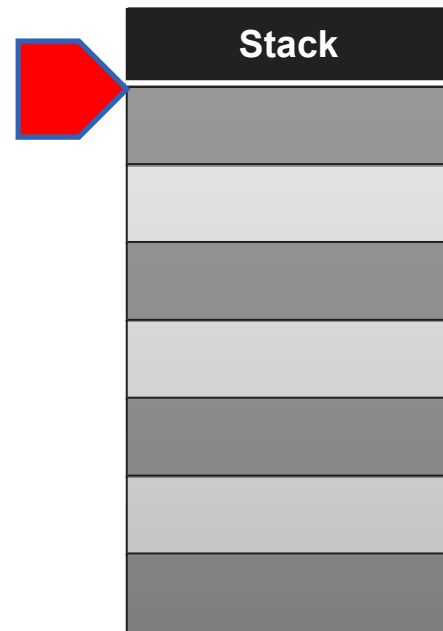
```
Output(name);
```

```
void Output(string arg)
{
    return;
}
```

[3640] Основной поток



Program.<Main>\$



Как работает Stack?

```
▶ string name = "Joe";
```

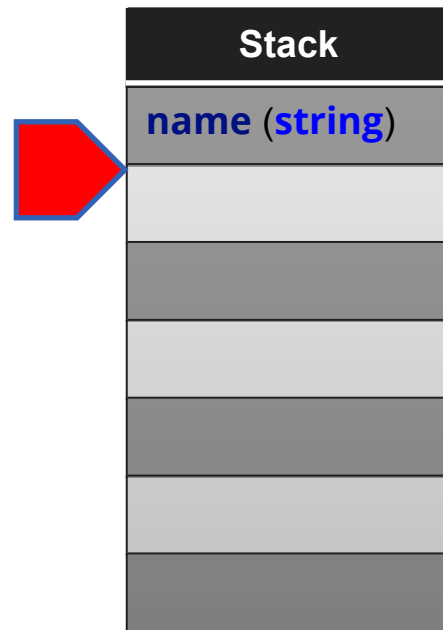
```
Output(name);
```

```
void Output(string arg)
{
    return;
}
```

[3640] Основной поток



Program.<Main>\$



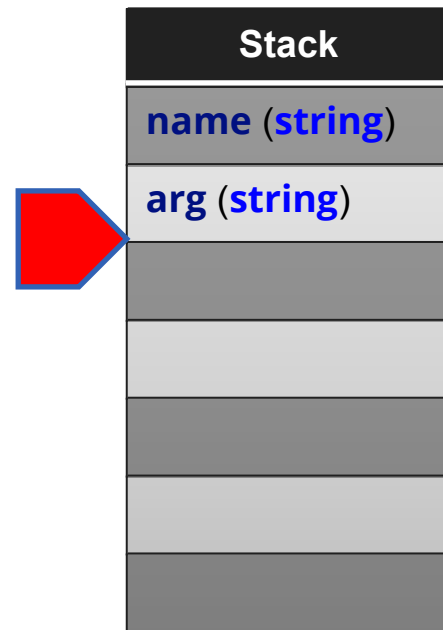
Как работает Stack?

```
string name = "Joe";
```

```
► Output(name);
```

```
void Output(string arg)  
{  
    return;  
}
```

[3640] Основной поток
➡ **Program.<Main>\$**

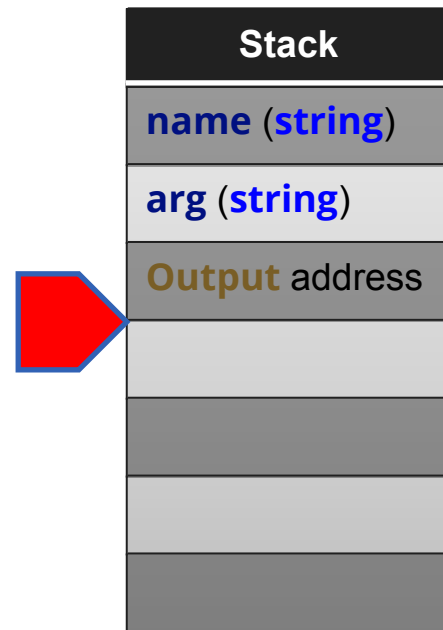
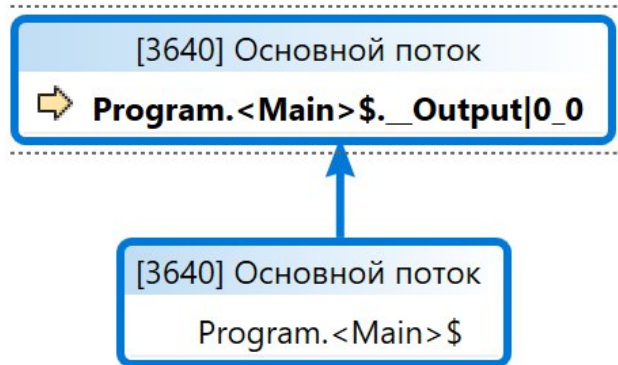


Как работает Stack?

```
string name = "Joe";
```

```
Output(name);
```

```
void Output(string arg)
{
    return;
}
```

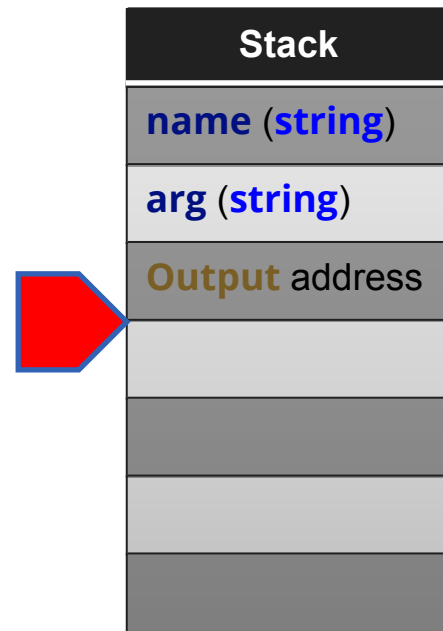
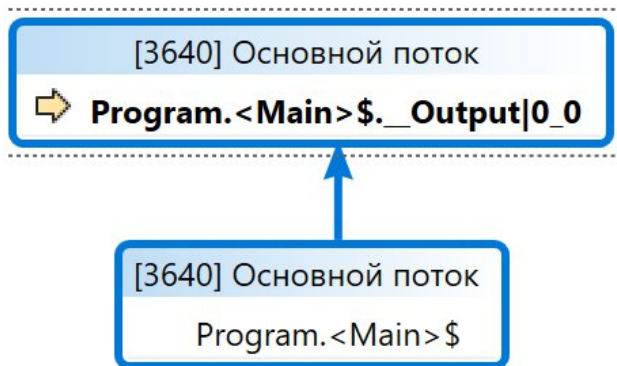


Как работает Stack?

```
string name = "Joe";
```

```
Output(name);
```

```
void Output(string arg)
{
    ►return;
}
```



Как работает Stack?

```
string name = "Joe";
```

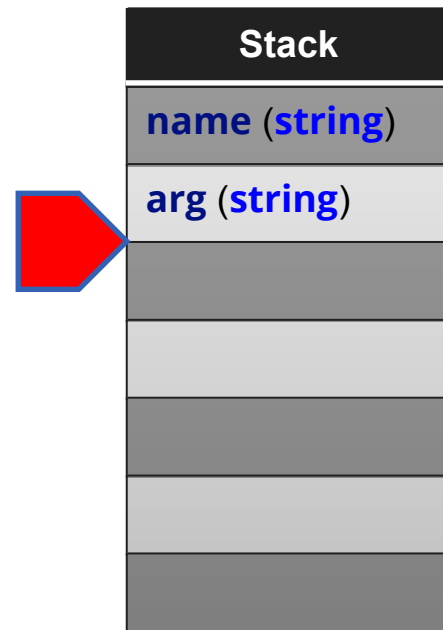
```
Output(name);
```

```
void Output(string arg)
{
    return;
}
```

[3640] Основной поток



Program.<Main>\$



Создание и размещение структур

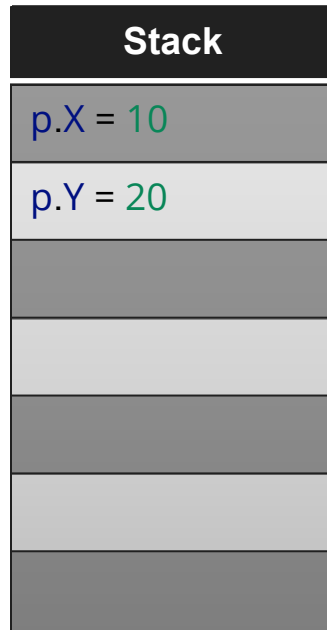
`struct` является значимым типом. Значения `struct` также хранятся в стеке.

Пример:

```
struct Point
{
    public int X;
    public int Y;
}
```

```
► Point p = new Point
{ X = 10, Y = 20 };
```

При передаче `p` в метод по значению — будет создана **копия всей структуры**.



Производительность стека

Стек работает по принципу LIFO (последним пришел — первым ушел), что делает доступ к нему крайне эффективным:

1. **Мгновенное выделение памяти.**

Чтобы выделить память на стеке, процессору достаточно просто изменить значение одного регистра — указателя стека (Stack Pointer). Это одна машинная инструкция.

2. **Кэш-дружелюбность.**

Стек занимает непрерывный блок памяти. Благодаря этому данные на стеке почти всегда находятся в L1/L2 кэше процессора. Скорость доступа к такому кэшу в десятки раз выше, чем к основной оперативной памяти.

3. **Прямая адресация.**

Доступ к локальным переменным происходит по фиксированному смещению относительно указателя кадра, что выполняется процессором очень быстро.

Live



Практика(смотрим стек в VS)

```
int a = 0x12345678;  
int b = 0x22335544;
```

```
Example strExample = new Example();  
strExample.a = 0x22222222;  
strExample.b = 0x11111111;
```

```
struct Example  
{  
    public int a;  
    public int b;  
}
```

Регистр	Адрес
RSP	0x000000C817D7EC90

```
0x000000C817D7EC60 68 ed d7 17 c8 00 00 00 08 ef d7 17 c8 00 00 00  
0x000000C817D7EC70 a0 e0 a0 a4 fa 7f 00 00 08 ef d7 17 c8 00 00 00  
0x000000C817D7EC80 68 ed d7 17 c8 00 00 00 b8 ed d7 17 c8 00 00 00  
0x000000C817D7EC90 22 22 22 22 11 11 11 11 44 55 33 22 78 56 34 12  
0x000000C817D7ECA0 68 ed d7 17 c8 00 00 00 08 ef d7 17 c8 00 00 00  
0x000000C817D7ECB0 e0 ec d7 17 c8 00 00 00 b3 9f 4f 04 fb 7f 00 00  
0x000000C817D7ECC0 c0 90 00 d1 79 02 00 00 e8 f2 d7 17 c8 00 00 00
```



Практика(смотрим стек в VS)

```
int a = 0x12345678;  
int b = 0x22335544;
```

Example strExample

```
    = new Example();  
strExample.a = 0x22222222;  
strExample.b = 0x33333333;
```

```
foo();
```

```
void foo() { int x = 0x11111111;}
```

```
struct Example
```

```
{  
    public int a;  
    public int b;  
}
```

Регистр	Адрес
RSP	0x000000618757E8A0

```
0x000000618757E840 70 e8 57 87 61 00 00 00 04 00 00 00 11 11 11 11  
0x000000618757E850 68 e9 57 87 61 00 00 00 08 eb 57 87 61 00 00 00  
0x000000618757E860 70 e8 57 87 61 00 00 00 dc df 87 31 fd 7f 00 00  
0x000000618757E870 d8 90 00 53 22 02 00 00 e8 ee 57 87 61 00 00 00  
0x000000618757E880 e8 ee 57 87 61 00 00 00 f9 ea 57 87 61 00 00 00  
0x000000618757E890 22 22 22 22 33 33 33 33 44 55 33 22 78 56 34 12  
0x000000618757E8A0 68 e9 57 87 61 00 00 00 08 eb 57 87 61 00 00 00  
0x000000618757E8B0 e0 e8 57 87 61 00 00 00 73 ef 3f 91 fd 7f 00 00  
0x000000618757E8C0 d8 90 00 53 22 02 00 00 e8 ee 57 87 61 00 00  
00
```

Shadow Space (специально резервируется для **Windows x64**)

△Значения регистров при вызове функций также попадают в стек:

```
0x000000618757E870 d8 90 00 53 22 02 00 00 e8 ee 57 87 61 00 00 00  
0x000000618757E880 e8 ee 57 87 61 00 00 00 f9 ea 57 87 61 00 00 00
```

Heap (Managed Heap): модели хранения и жизненный цикл объектов



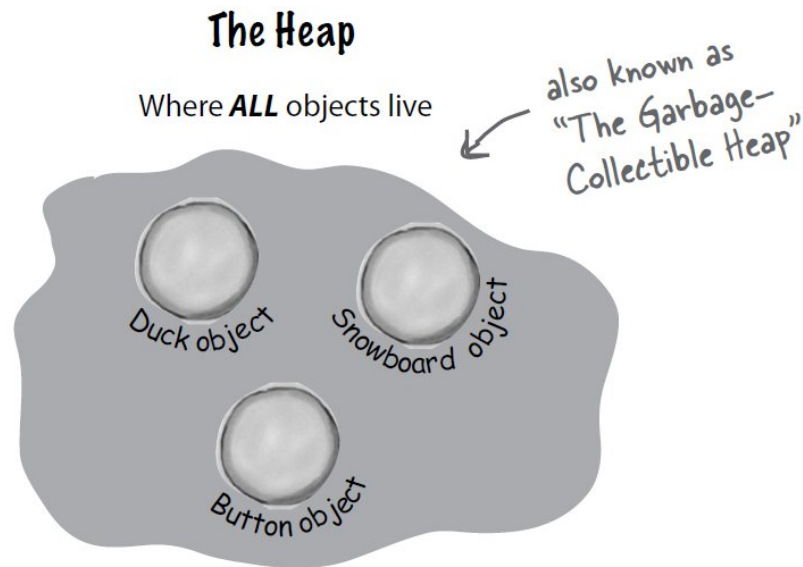
Что такое куча?

Куча — это область памяти, где хранятся все ссылочные типы ([class](#), [array](#), [delegate](#), [string](#) и т. д.).

Она используется для выделения памяти в **«произвольных»** количествах и может использоваться для создания объектов, которые живут дольше, чем время выполнения функции.

При переполнении кучи будет брошено исключение [OutOfMemoryException](#)

Куча растет в сторону увеличения адресов.



Жизненный цикл объектов в куче

Жизненный цикл объектов в куче:

1. Аллокация (процесс выделения памяти). Сдвигается указатель и обнуляются биты.
2. Размещение
3. Фрагментация и компактизация (этап сборки мусора)

Аллокация

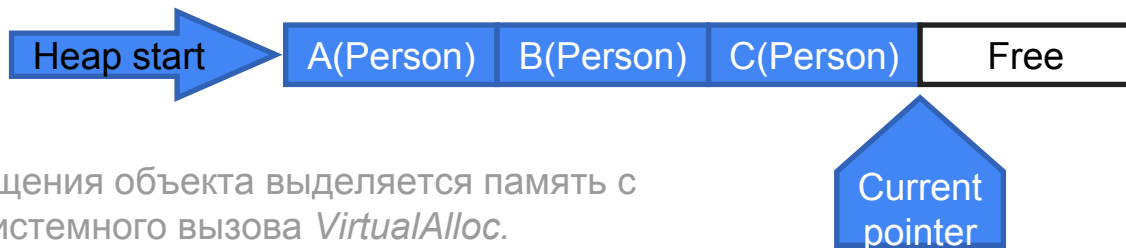
Что делает оператор `new`:

1. Вычисляет количество байтов под размер объекта.
2. Выделяет нужное количество байтов в куче.
3. Инициализирует указатель на объект-тип и на блок синхронизации.
4. Вызывает конструктор экземпляра типа.

```
var person = new Person();
```

Этапы:

1. CLR видит, что `Person` — ссылочный тип.
2. Выделяется память.
3. Аллокатор просто прибавляет смещение к текущему указателю "свободной памяти".



*Для размещения объекта выделяется память с помощью системного вызова *VirtualAlloc*.

Определение размера объекта и его адреса

```
class Person
{
    byte some = 1;
    int age = 10;
    string name = "Bob";
}
```

Каждый объект в .NET:

Имеет заголовок (Object Header, 8–16 байт), где:

- sync block (для lock);
- указатель на объект-тип (Type handle / MethodTable pointer);

Затем идут данные объекта.

Адресом объекта считается адрес начала заголовка, т.е. `this` в C# указывает на этот адрес.

Object Header	Адрес (например, 0x0010A000)
Sync block (4 B)	используется при lock (obj)
Type pointer (4/8 B)	указатель на MethodTable (тип Person)
some (byte) (1 байт)	значение 1
padding (3 байта)	выравнивание (до 4/8) байт
age (int) (4 байта)	значение 10
name(string)	ссылка на строку "Bob" в куче



Размещение объекта в памяти

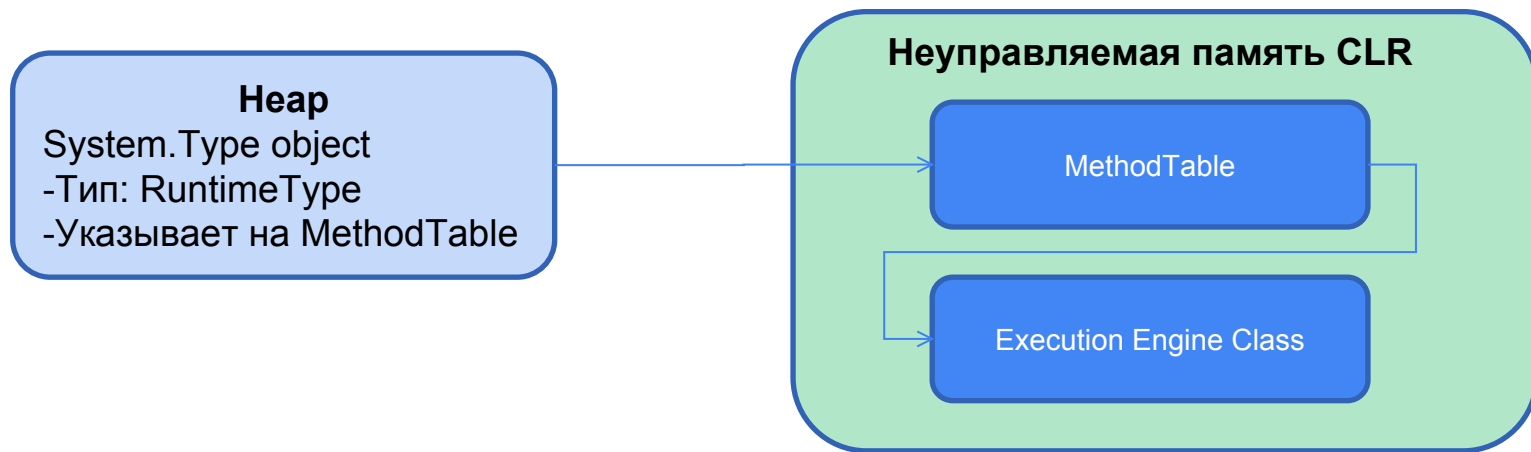
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0		Sync block				Type pointer				1	Padding				0	0	0	10	0x0010A00D			
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

```
class Person
{
    byte some = 1;
    int age = 10;
    string name = "Bob";
}
```

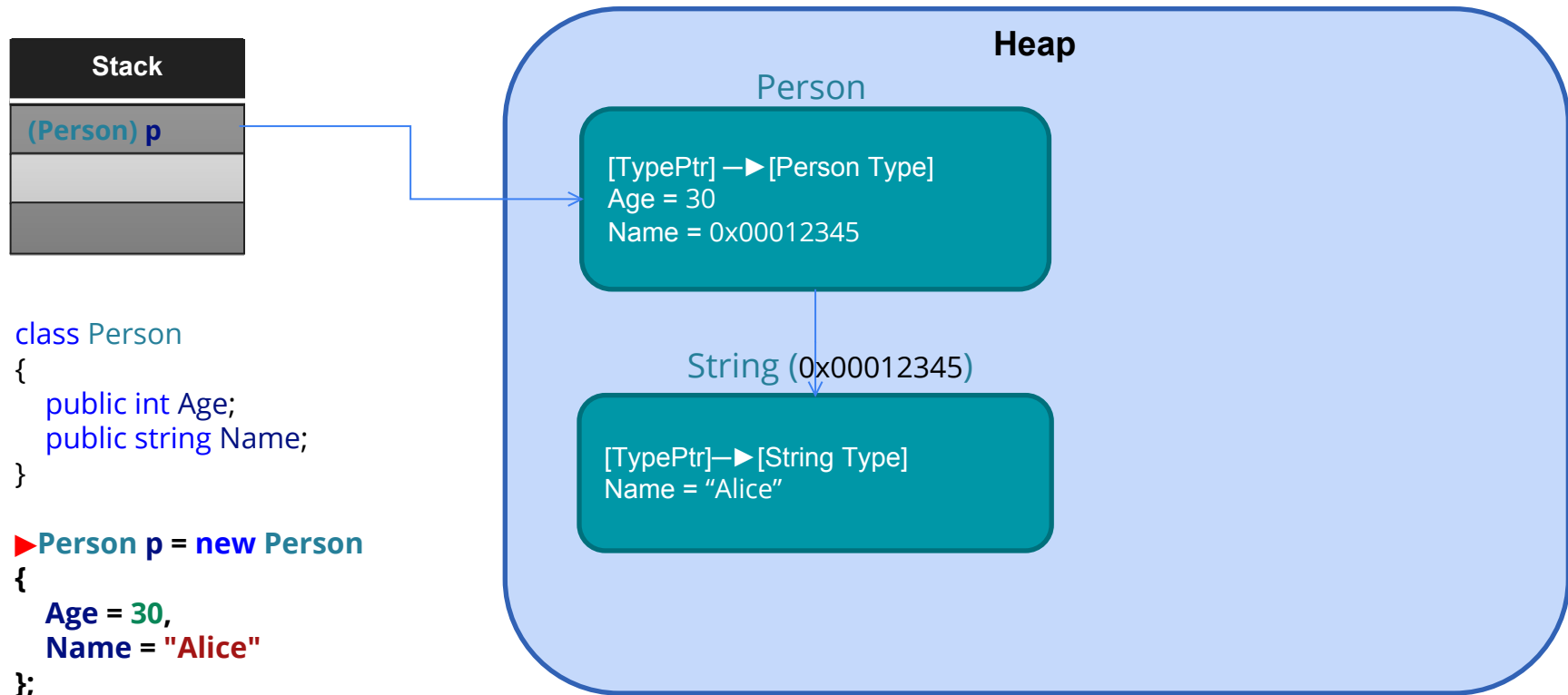


Что такое объект-тип?

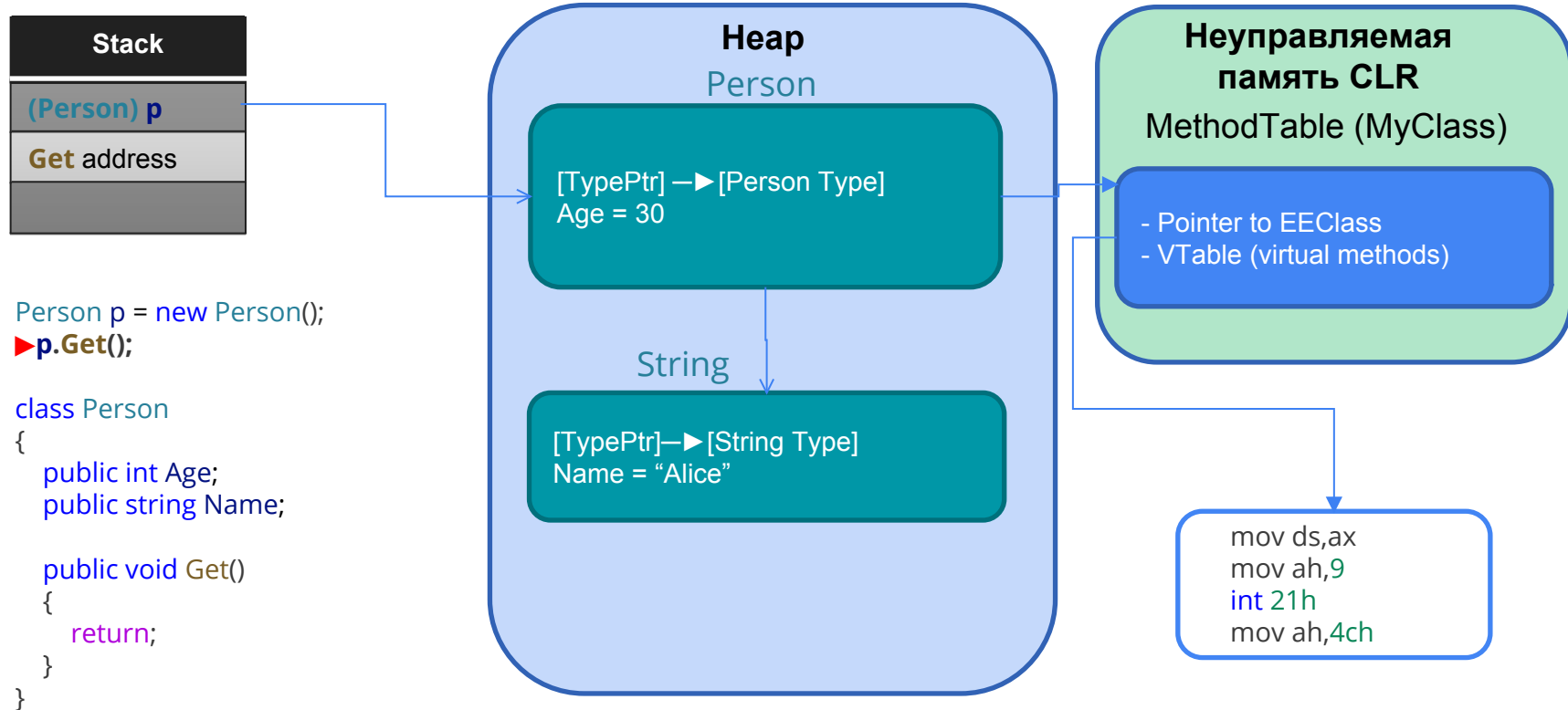
Объект-тип – **метаданные и структура, которые описывают конкретный тип** (класс, структура, интерфейс и т. д.) и всё, что нужно *runtime* для его работы.



Представление объекта в куче



Вызов метода объекта в куче



Что такое MethodTable?

Это структура, которая есть у **каждого типа**, и она напрямую связана с объектом в куче:

- Она содержит указатель на EEClass (метаданные типа).
- GC-дескриптор (layout полей, какие поля — ссылочные, какие — значимые).
- Размер объекта в памяти.
- Таблица виртуальных методов (vtable): указатели на JIT-компилированные функции.
- Указатель на базовый MethodTable (для наследования).
- Флаги: является ли тип абстрактным, интерфейсом, содержит ли finalizer и т.д.

Что такое EEClass?

EEClass хранит больше логической информации о типе, которую можно сопоставить с метаданными сборки:

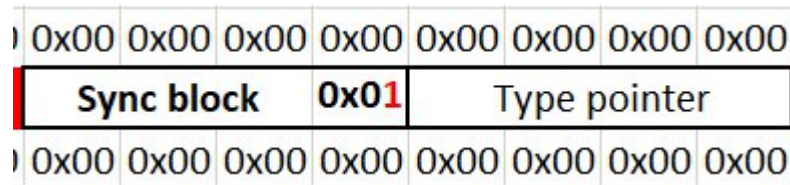
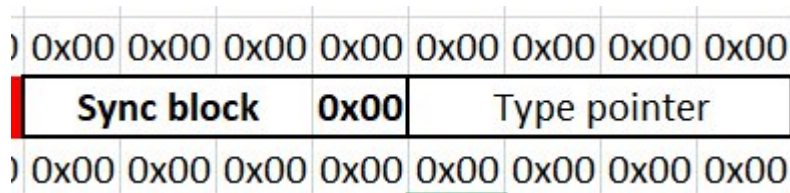
- Имя класса (ссылка на строку из метаданных).
- Пространство имён.
- Базовый класс (EEClass* на родителя).
- Интерфейсы (список EEClass* интерфейсов).
- Поля: имена, типы, смещения в объекте, модификаторы (static, readonly, volatile)
- Методы: сигнатуры, атрибуты, RVA (ссылка на IL-код в PE-файле).
- Атрибуты (например, [Serializable], [Obsolete] и т.п.).
- Информация для Reflection.

Блок синхронизации

В .NET **блок синхронизации (Sync Block)** — это механизм, который позволяет любому объекту (ссылочного типа) выступать в роли «замка» ([lock](#)) для управления многопоточностью.

По умолчанию: Это поле равно 0. Это значит, что объект пока не используется для синхронизации.

```
► lock (example)
{
    example.Show();
}
```



Что такое выравнивание?

Выравнивание - это способ размещения данных в оперативной памяти, при котором начальный адрес переменной кратен её размеру или установленному размеру «слова» процессора.

Простыми словами: это правило, согласно которому данные должны начинаться с «правильных» ячеек памяти, чтобы процессору было легче и быстрее их читать.

Зачем нужно выравнивание?

Процессоры обычно читают память не по одному байту, а блоками (чаще всего по 4 или 8 байт).

1. **Выровненные данные:** Если 4-байтовое целое число (`int`) лежит по адресу, кратному 4 (например, 0, 4, 8), процессор считывает его за **одну операцию**.
2. **Невыровненные данные:** Если это же число начнется, например, с адреса 3, оно окажется «разбито» между двумя блоками памяти. Процессору придется сделать **два чтения**, а затем программно склеивать результат. Это замедляет работу.

```
struct Example
{
    char a;  // 1 байт
    // Здесь компилятор вставит 3 байта Padding
    int b;   // 4 байта
}
```

Без выравнивания структура занимала бы 5 байт. С выравниванием она займет **8 байт**, зато доступ к переменной `b` будет мгновенным.

Производительность кучи

Куча — это общая область памяти, управление которой требует сложных алгоритмов:

1. Сложное выделение.

При создании объекта в куче (например, через [new](#)), системе нужно найти свободный блок подходящего размера. Это требует просмотра структур данных (списков свободных блоков) и работы менеджера памяти.

2. Косвенная адресация (Indirection).

В куче вы храните не сам объект, а ссылку (указатель) на него. Чтобы прочитать данные, процессору нужно сначала прочитать адрес, а затем совершить «прыжок» по этому адресу. Это увеличивает время доступа.

3. Пропахи кэша (Cache Misses).

Объекты в куче могут быть разбросаны по разным адресам. Из-за этого процессору сложнее предсказать, какие данные понадобятся следующими, и данные реже попадают в быстрый кэш.

4. Сборка мусора (GC).

Работа с кучей включает дополнительные расходы на периодическую остановку программы для очистки неиспользуемых объектов и дефрагментации памяти.

Live



Практика(смотрим кучу в VS)

```
Example example = new Example();  
example.a = 0x22222222;  
example.b = 0x11111111;
```

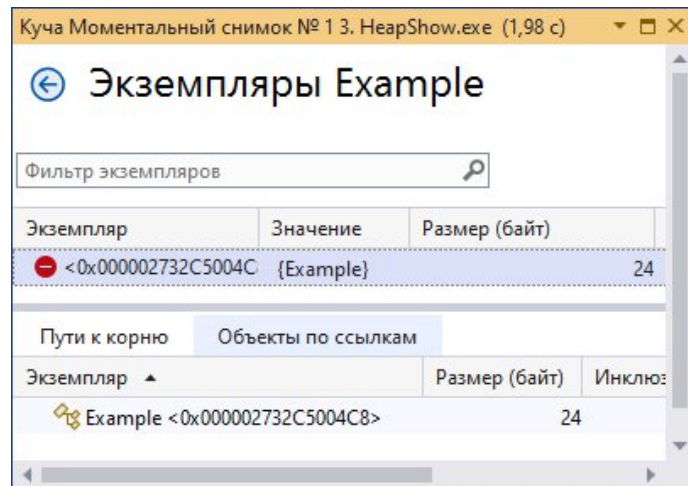
```
lock(example)  
{ example.Show(); }
```

```
class Example  
{  
    public int a;  
    public int b;  
    public void Show() { }  
}
```

```
0x0000002732C5004B8 ff ff ff ff ff ff 00 00 00 00 01 00 00 00  
0x0000002732C5004C8 e0 ec f9 a4 fa 7f 00 00 22 22 22 22 11 11 11 11  
0x0000002732C5004D8 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Регистр	Адрес
RSP	0x000000E96D79E920

```
0x000000E96D79E910 20 e9 79 6d e9 00 00 00 b2 a2 c6 a4 fa 7f 00 00  
0x000000E96D79E920 c8 04 50 2c 73 02 00 00 e0 04 50 2c 73 02 00 00  
0x000000E96D79E930 98 ef 79 6d e9 00 00 00 a9 eb 79 6d e9 00 00 00
```



boxing и unboxing: скрытые аллокации и влияние на производительность



Что такое boxing?

Boxing — это процесс **упаковки** значения* (например, `int`, `bool`, `struct`) в **объект в куче** (heap), чтобы с ним можно было обращаться как с `object`.

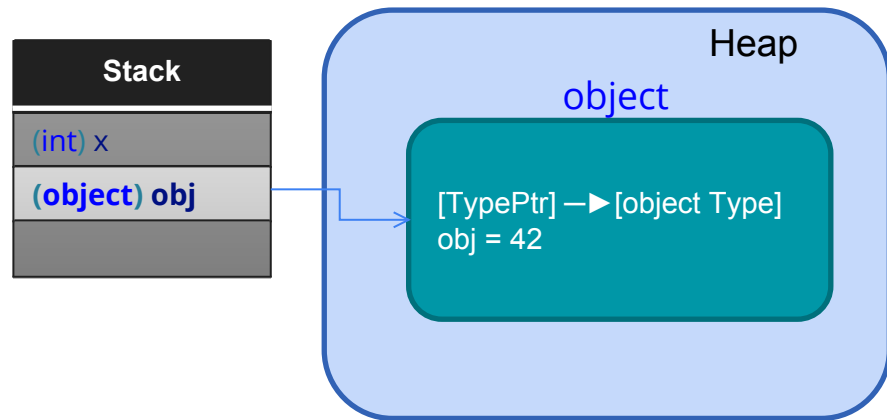
**ссылочные объекты — всегда «упакованные»*

Пример:

```
int x = 42;  
object obj = x; // boxing
```

Что происходит:

1. Значение `x` копируется.
2. Создаётся новый объект в куче (heap).
3. В него копируется значение `x`.
4. Переменная `obj` хранит ссылку на этот объект.



Что такое unboxing?

Unboxing — это **извлечение** значения из объекта обратно в значимый тип.

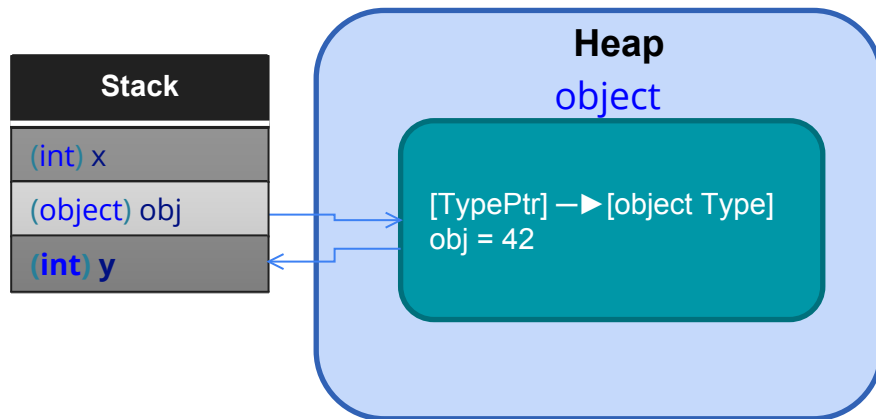
Пример:

```
int x = 42;  
object obj = x; // boxing
```

```
int y = (int)obj; // unboxing
```

Что происходит:

1. CLR проверяет, что `obj` — это действительно упакованный `int`.
2. Извлекается адрес полей из объекта
3. Значение **копируется** в стек в переменную `y`.



MS IL

С# код	IL код
<code>int x = 42;</code>	<code>ldc.i4.s 42 stloc.0</code>
<code>// Упаковка</code> <code>object obj = x;</code>	<code>ldloc.0 box [System.Private.CoreLib]System.Int32 stloc.1</code>
<code>// Распаковка</code> <code>int y = (int)obj;</code>	<code>ldloc.1 unbox.any [System.Private.CoreLib]System.Int32 stloc.2</code>

1. `box [mscorlib]System.Int32`
 - Создаёт объект в куче.
 - Копирует значение `int` из стека в выделенную память.
 - Возвращает ссылку на объект (типа `object`).
2. `unbox.any [mscorlib]System.Int32`
 - Проверяет, что ссылка действительно указывает на объект `System.Int32`.
 - Копирует значение обратно в стек (в переменную `int`).

Использование интерфейсов со **struct**

```
InvertValue invertValue = new InvertValue { X = 2 };
```

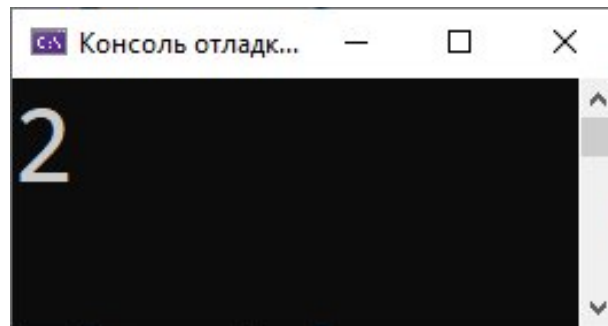
```
((IInvert)invertValue).Invert();
```

//Вопрос - что будет выведено в консоль?

```
Console.WriteLine($"{invertValue.X}");
```

```
interface IInvert  
{ void Invert(); }
```

```
struct InvertValue : IInvert  
{  
    public int X;  
    public void Invert()  
    {  
        this.X *= (-1);  
    }  
}
```



Вызов виртуальных методов **object**

При обращении к методам (`ToString()`, `GetHashCode()`, `Equals()`) **базового** типа упаковка нужна.

```
Point p = new Point { X = 1, Y = 2 };  
string s = p.ToString(); // boxing, так как ToString() не переопределён
```

```
struct Point  
{ public int X, Y; }
```

При обращении к **переопределённым** методам не нужна упаковка.

```
Point p = new Point { X = 1, Y = 2 };  
string s = p.ToString(); //нет boxing, так как ToString() переопределён
```

```
struct Point  
{  
    public int X, Y;  
    public override string ToString() {return string.Empty;}  
}
```


Непараметризованные коллекции

Одним из наиболее частых примеров неявной упаковки служит использование необобщенных коллекций.

```
ArrayList list = new ArrayList();
```

```
// Добавляем значения int
for (int i = 0; i < 5; i++)
{
    // ● упаковка каждого int
    list.Add(i);
}
```

```
// Достаём обратно
foreach (object obj in list)
{
    // ● распаковка (unboxing)
    int value = (int)obj;
}
```

```
List<int> list = new List<int>();
```

```
// Добавляем значения int
for (int i = 0; i < 5; i++)
{
    // ✓ нет упаковки
    list.Add(i);
}
```

```
// Достаём обратно
foreach (int value in list)
{
    // ✓ нет распаковки
    Console.WriteLine(value);
}
```

Влияние на производительность

Процессы упаковки/распаковки снижают производительность, так как:

1. Происходит трата процессорного времени на аллокацию в куче и копирование в кучу данных.
2. Извлечение данных из кучи происходит медленнее, чем из стека.
3. Дополнительные аллокации учащают сборку мусора.

Основы работы Сборщика мусора (Garbage Collector, GC)



Необходимость сбора мусора

В памяти приложения мы все время создаем новые объекты, через какое-то время некоторые из них нам становятся не нужны.

Данные объекты являются «мусором». При этом память не бесконечна, расходовать безлимитно её нельзя. С другой стороны нельзя очищать память, занятой чем-то полезным (тем что планируется использовать).

Какой выход – убирать то, что уже не нужно.

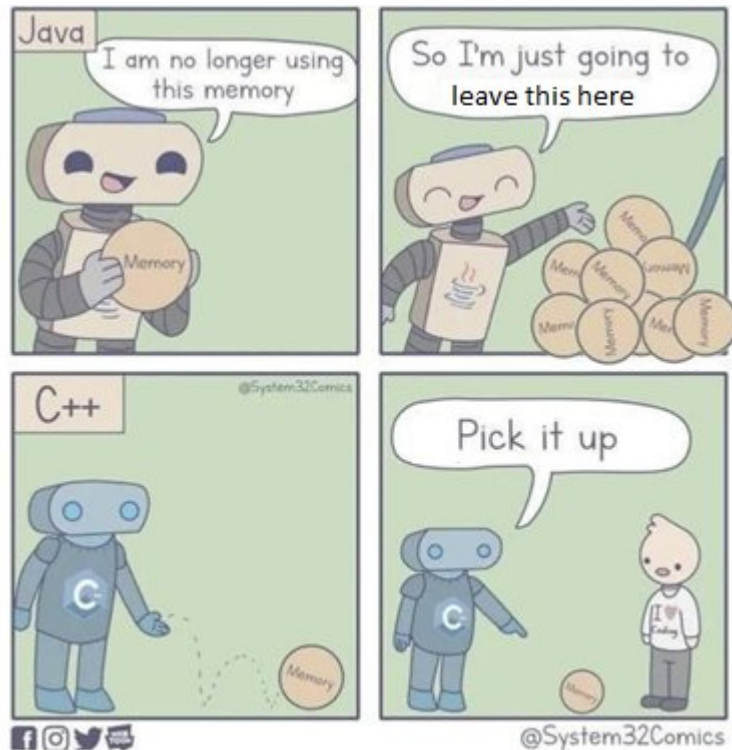


Что такое GC(Garbage Collection)?

В C++ существовала проблема утечки памяти, за её очищением программисты должны были следить самостоятельно. В CLR этим занимается сборщик мусора (Garbage Collection, GC).

Запуск уборки мусора происходит в следующих случаях:

1. Если заполнено поколение 0.
2. Если Windows сообщила о нехватке памяти.
3. При выгрузке домена приложения.
4. При запуске `GC.Collect()`.
5. Завершение работы CLR. При этом ОС очищает всю память.

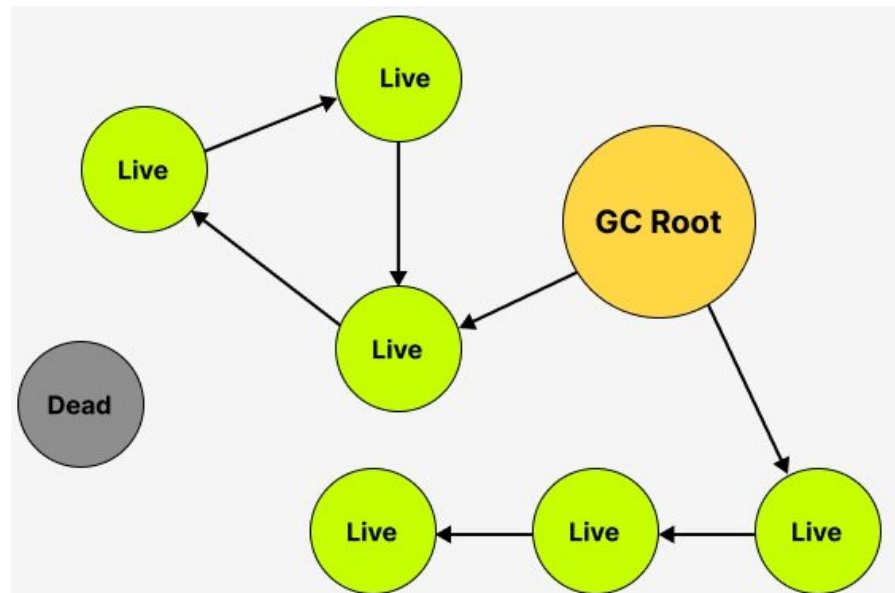


Алгоритм работы сборки мусора



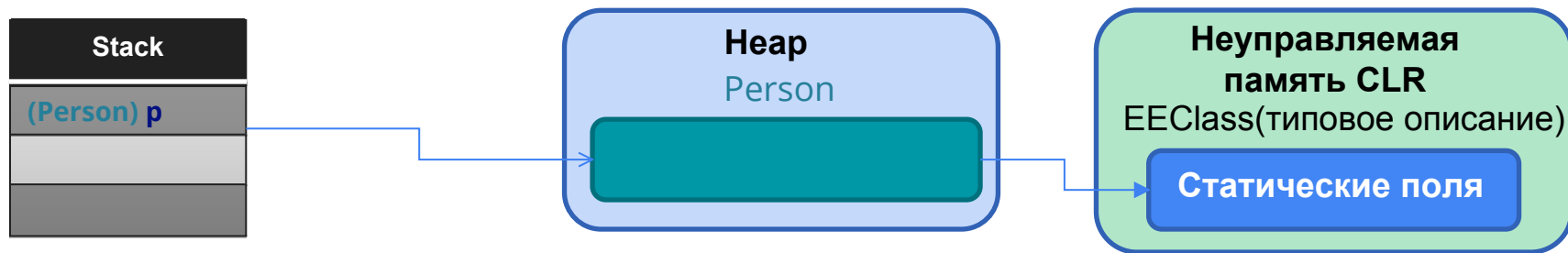
Алгоритм сборки мусора

1. Остановка всех потоков приложения.
2. Поиск **корней**.
3. Все объекты, ссылки на которые найдены, помечаются как живые.
4. Объекты, не отмеченные как живые, удаляются.
5. Живые объекты перемещаются ближе друг к другу, их ссылки переписываются на новые адреса.
6. Запуск потоков приложения.



Корни

Корни - переменные стека ссылочного типа, также статичные поля, которые ссылаются на объекты в куче. Это стартовые точки, от которых сборщик мусора начинает обход памяти, чтобы найти *живые объекты*.



```
class Person
{
    public static object myObject;
    public void Get()
    {
        return;
    }
}
```


Маркировка

В CLR у каждого сегмента кучи (будь то сегмент Gen0, Gen1, Gen2 или LOH) есть **своя карта занятости и карта живых объектов**, и биты «живости» (mark bits) хранятся не в самих объектах, а в **отдельных служебных структурах GC**, которые находятся рядом с сегментом, но **вне управляемой памяти**.

Память, занятая объектами до сборки мусора

A	B	Cx	D	Ex	Fx	G
---	---	----	---	----	----	---

BitMap: 1 1 0 1 0 0 1

Фрагментация

Фрагментация — это ситуация, когда в памяти много **свободных участков**, но они разрознены, и нет одного большого непрерывного блока, чтобы разместить крупный объект.

Память, занятая объектами до сборки мусора

A	B	C	D	E	F	G
---	---	---	---	---	---	---

Память, занятая объектами после сборки мусора

A	B		D		F	
---	---	--	---	--	---	--

Компактизация

Компактизация — это процесс, при котором сборщик мусора (GC) в .NET **перемещает живые объекты в памяти вплотную друг к другу**, чтобы убрать «дыры» (фрагментацию) и освободить один большой непрерывный блок.

Зачем:

1. Сокращается рабочий размер приложения.
2. Свободное состояние становится непрерывным. Мы точно будем знать - сколько памяти у нас есть.

Память, занятая объектами до сборки мусора

A	B		D		F	
---	---	--	---	--	---	--

Память, занятая объектами после сборки мусора

A	B	D	F			
---	---	---	---	--	--	--

**Сразу после сжатия продолжать работу приложения нельзя. Ссылки на все объекты в куче - перезаписываются.*



Поколения сборки мусора



Поколения сборки мусора

Принципы сборки мусора:

1. Чем **младше** объект, тем **короче** его время жизни
2. Чем **старше** объект, тем **длиннее** его время жизни
3. Уборка мусора **в части** кучи выполняется **быстрее**, чем во всей куче.

Сборщик мусора CLR имеет 3 поколения: 0, 1, 2.

Меньшее всего живут объекты из 0 поколения.

Наиболее долгоживущие - во 2.

Managed Heap
Generation 0
Generation 1
Generation 2

Очередность сборки мусора в поколениях

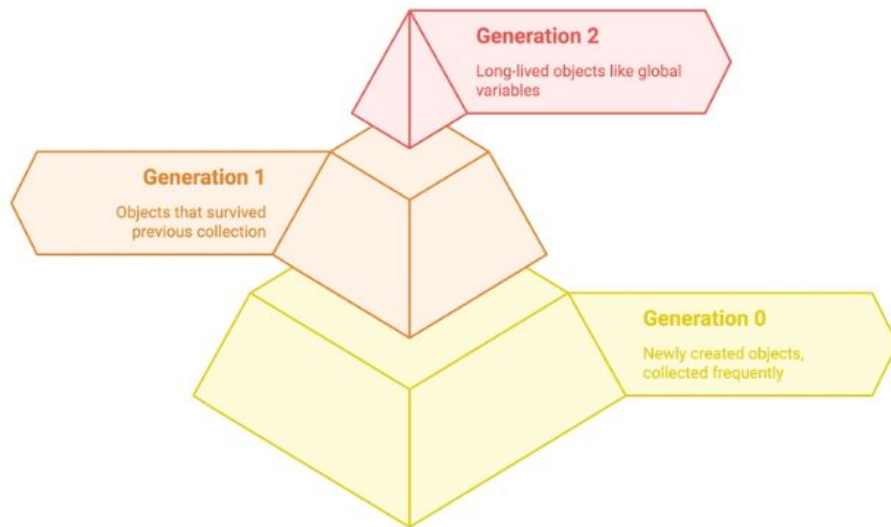
Общий сценарий:

1. При иницировании сборки мусора GC просматривает состояние поколений, начиная с самого верхнего(N).
2. Если N-е поколение переполнено, то происходит сборка мусора в нем и во всех нижележащих поколениях.
3. Если не переполнено, то спускаемся на поколение ниже и повторяем с п.2.

При сборке поколения 1 - собирается мусор и 0, и 1 поколения.

При сборке 2 поколения - 0, 1, 2. Сборка поколения 2 – **FullGC**.

Object Lifecycle in Garbage Collection



Порядок сборки мусора в поколениях



Вид кучи сразу после инициализации: все объекты в ней относятся к поколению 0, уборка мусора еще не выполнялась



Вид кучи после одной уборки мусора: выжившие объекты из поколения 0 переходят в поколение 1, поколение 0 пустует



В поколении 0 появились новые объекты, в поколении 1 — мусор

 - поколение 2;  - поколение 1;  - поколение 0.

Порядок сборки мусора в поколениях



Вид кучи после двух операций уборки мусора: выжившие объекты из поколения 0 переходят в поколение 1 (увеличивая его размер), поколение 0 пустует



В поколении 0 созданы новые объекты, количество мусора в поколении 1 увеличилось



Вид кучи после трех операций уборки мусора: выжившие объекты из поколения 0 переходят в поколение 1 (увеличивая его размер); поколение 0 пустеет

■ - поколение 2; ■ - поколение 1; ■ - поколение 0.

Порядок сборки мусора в поколениях



Новые объекты размещены в поколении 0, в поколении 1 появилось больше мусора



Вид кучи после четырех операций уборки мусора: выжившие объекты из поколения 1 переходят в поколение 2, выжившие объекты из поколения 0 переходят в поколение 1, поколение 0 снова пусто

■ - поколение 2; ■ - поколение 1; ■ - поколение 0.

Итого

Выделение памяти под процесс:

1. Каждая штатная сборка мусора начинается при заполнении поколения 0.
2. Если в более высоком поколении происходит сборка мусора, то она происходит во всех более низких.
3. Конец жизни объекта не означает его моментальное удаление из кучи.

Large Object Heap (LOH)



Что такое Large Object Heap (LOH)?

Large Object Heap (LOH) — это специальная область в управляемой куче (Managed Heap), предназначенная для **больших объектов**.

Эта память физически размещена отдельно от основной кучи.

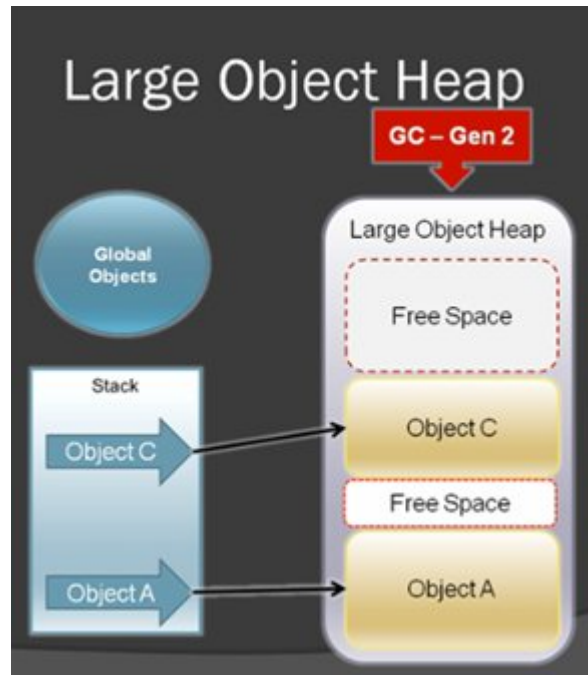
Что считается "большим объектом"?

Любой объект, который **занимает $\geq 85\,000$ байт**, попадает в LOH. Объем **85 000** жестко прописан в .net.

Примеры:

Большой массив байтов: `new byte[100_000]`

Большой массив структур: `new Point[50_000]`



Где LOH находится?

В памяти LOH — это **часть общей управляемой кучи (Managed Heap)**.

Адресно — LOH идёт отдельным "сегментом" памяти, у GC для неё отдельный указатель конца кучи. При этом LOH считается частью поколения 2.

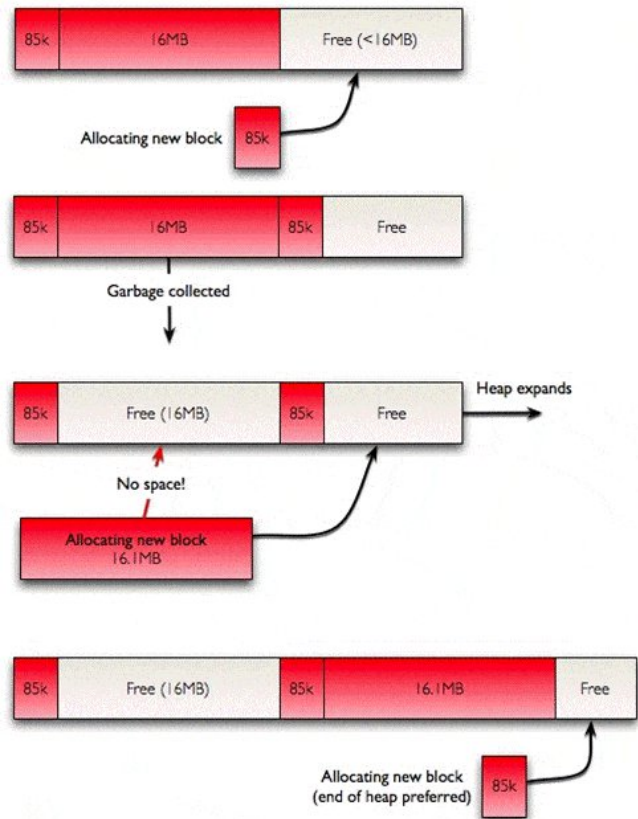
Это означает, что они не участвуют в сборке мусора G0 или G1, только в полной GC (**Full GC**).

Managed Heap
Generation 0
Generation 1
Generation 2
Large Object Heap (LOH)

Особенности LON

Сильная фрагментация LON

Большие объекты не перемещаются в обычных GC. Из-за этого LON со временем сильно фрагментируется.



Особенности LOH

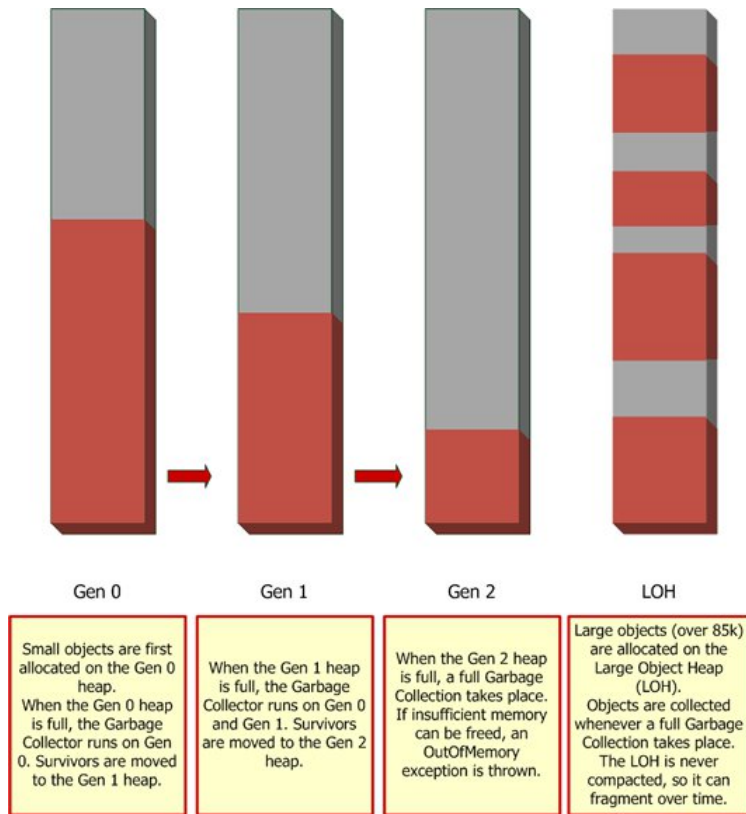
Компактификация LOH

LOH (Large Object Heap): по умолчанию **не компактифицируется**, чтобы избежать дорогих перемещений больших объектов. Объекты в LOH остаются на своих местах.

Компактификацию LOH можно вручную включить начиная с .NET 4.5.1+ :

//LOH будет компактифицирована при ближайшей FullGC

```
GCSettings.LargeObjectHeapCompactionMode =  
GCLargeObjectHeapCompactionMode.CompactOnce  
;  
GC.Collect();
```



Особенности LON

Проблем **LON (Large Object Heap)** в .NET:

1. Фрагментация памяти.

По умолчанию LON долго не компактизируется → между большими объектами появляются «дыры», которые нельзя использовать под мелкие объекты.

2. Долгий GC.

Объекты LON живут в Gen2, а сборка Gen2 — самая «тяжёлая» и редкая.

3. Большая стоимость выделения.

Выделение большого блока требует поиска подходящего свободного места в LON, что может быть медленнее, чем в обычной куче. При размещении нового объекта GC пытается впихнуть объект в подходящую по размеру область из освободившейся.

4. Риск `OutOfMemoryException` при наличии свободной памяти.

Из-за фрагментации может не оказаться непрерывного блока нужного размера, даже если общая свободная память есть.

Pinned Object Heap (POH)



Что такое POH?

POH (Pinned Object Heap) — это специальный *под-хил* в управляемой куче CLR (.NET 5+), выделенный для **закреплённых (pinned)** объектов.

Закреплённый объект — это объект, адрес которого CLR не имеет права перемещать (например, при работе с небезопасным кодом или при передаче буфера в нативные API).

Обычные объекты перемещаются сборщиком мусора (GC) во время компактизации, чтобы избежать фрагментации. Но pinned-объекты препятствуют этому → поэтому их выделяют в отдельной области (POH).



Зачем нужен РОН?

В .NET сборщик мусора — «кочующий». Чтобы не было дыр в памяти (фрагментации), он двигает объекты, прижимая их друг к другу.

Когда мы передаем данные в нативный код (C++ библиотеку или API Windows), мы даем ему **адрес** (указатель).

Если GC передвинет объект, пока C++ с ним работает, указатель станет невалидным, и программа упадет ([Access Violation Exception](#)).



Фиксация объекта

Старый синтаксис

```
// 1. Создаем обычный массив (в обычной куче)
int[] commonArray = new int[1024];

// 2. Временно закрепляем его
unsafe
{
    fixed (int* ptr = commonArray)
    {
        // Пока мы внутри этого блока, GC
        // запрещено двигать этот массив.

        // Как только мы выйдем за скобки,
        // массив снова может быть перемещен.
        DoNativeWork((IntPtr)ptr);
    }
}
```

Новый синтаксис

```
// 1. Объект сразу рождается "закрепленным"
// в специальной куче POH
int[] pinnedArray = GC.AllocateArray<int>
    (1024, pinned: true);

// 2. Теперь можно безопасно получить адрес
// и передать в нативный код
unsafe
{
    fixed (int* ptr = pinnedArray)
    {
        // ptr гарантированно не изменится
        // и GC не будет страдать от фрагментации
    }
}
```

Свойства РОН

Объекты НЕ перемещаются: С момента создания и до смерти объект в РОН стоит на одном и том же адресе.

ГС всё равно работает: Важный миф: «РОН не чистится». **Это неправда.** ГС удаляет объекты из РОН, когда они больше не нужны, но он не делает дефрагментацию (сжатие) в этой области.

Снижение пауз: Благодаря выносу таких объектов в отдельную кучу, основной ГС (в 0, 1 и 2 поколениях) работает быстрее, так как ему не нужно «обтекать» неподвижные объекты.



Ваши вопросы



если есть вопросы



если вопросов нет

